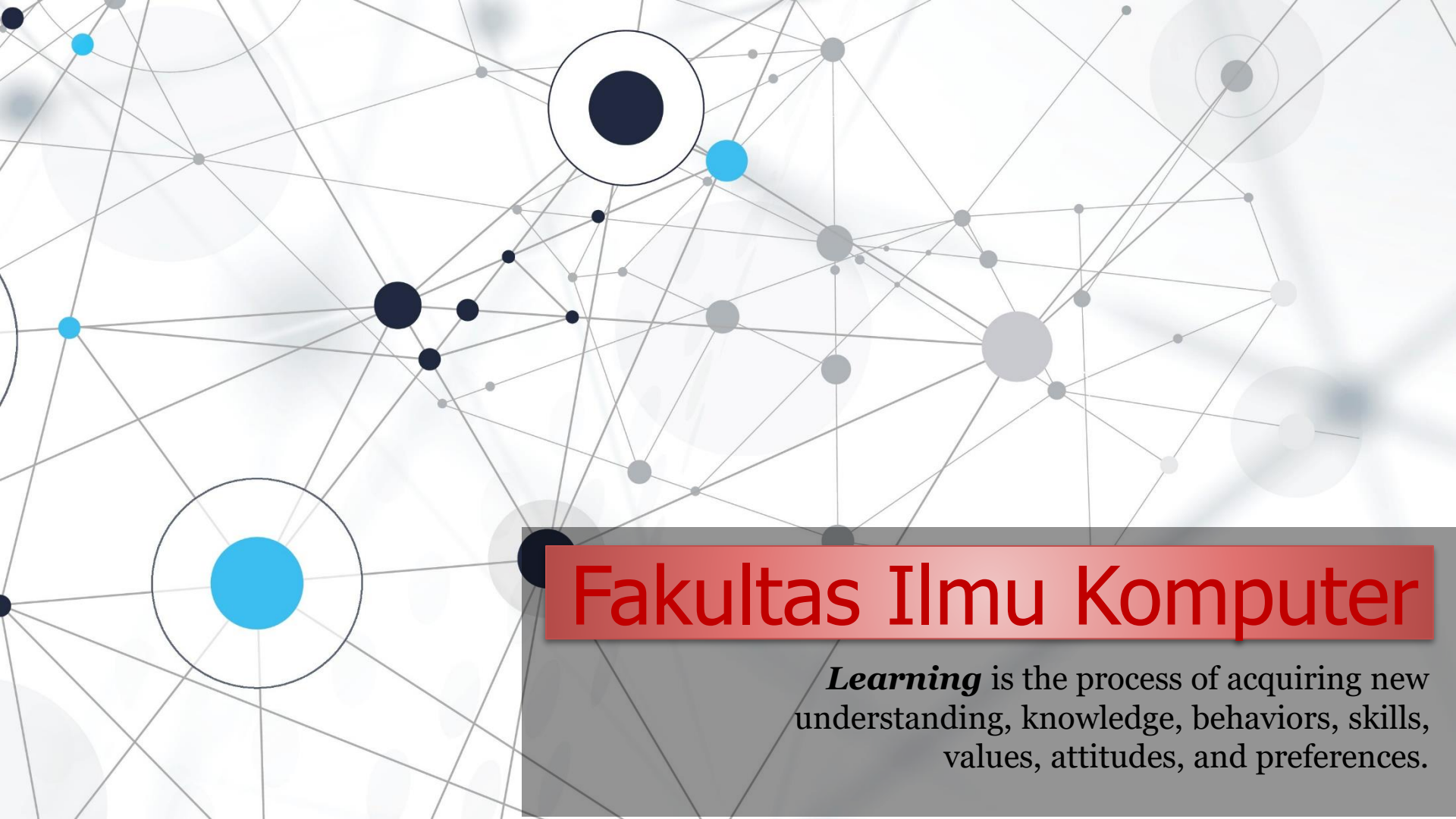


An abstract network diagram with various sized nodes (black, blue, grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

OOP in Python – Polymorphism

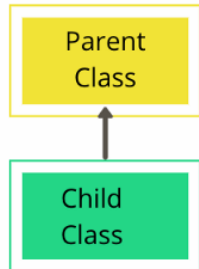


**Welcome to *the* Object Oriented
Programming *course***

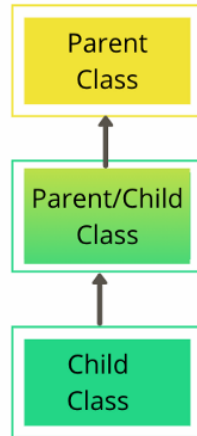
Parent Class vs Child Class

Parent class is the class being inherited from, also called base class. Child class is the class that inherits from another class, also called derived class.

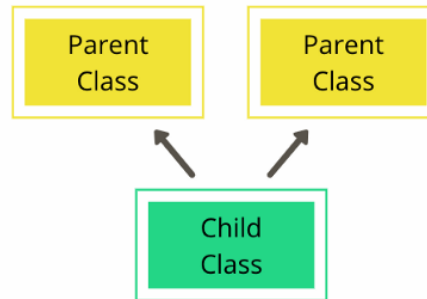
Simple(Single)
Inheritance



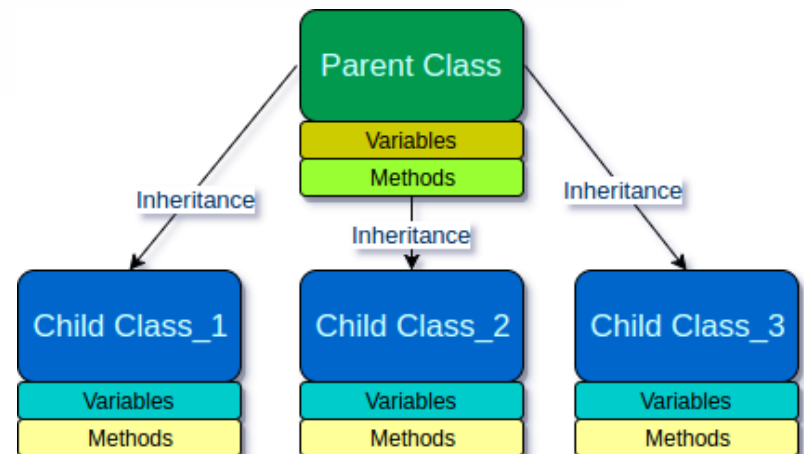
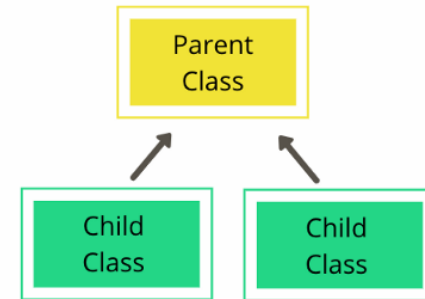
Multi Level
Inheritance

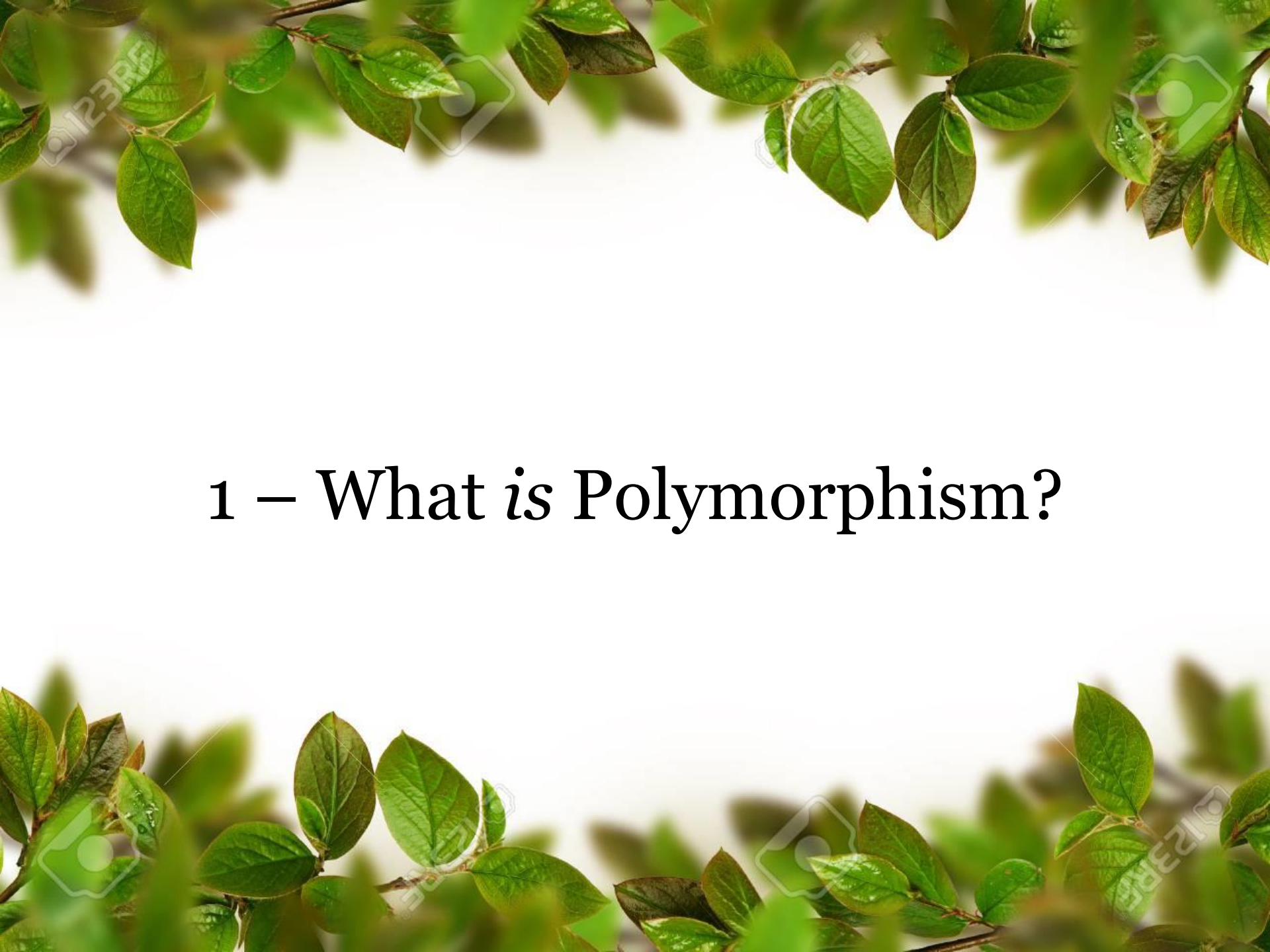


Multiple
Inheritance



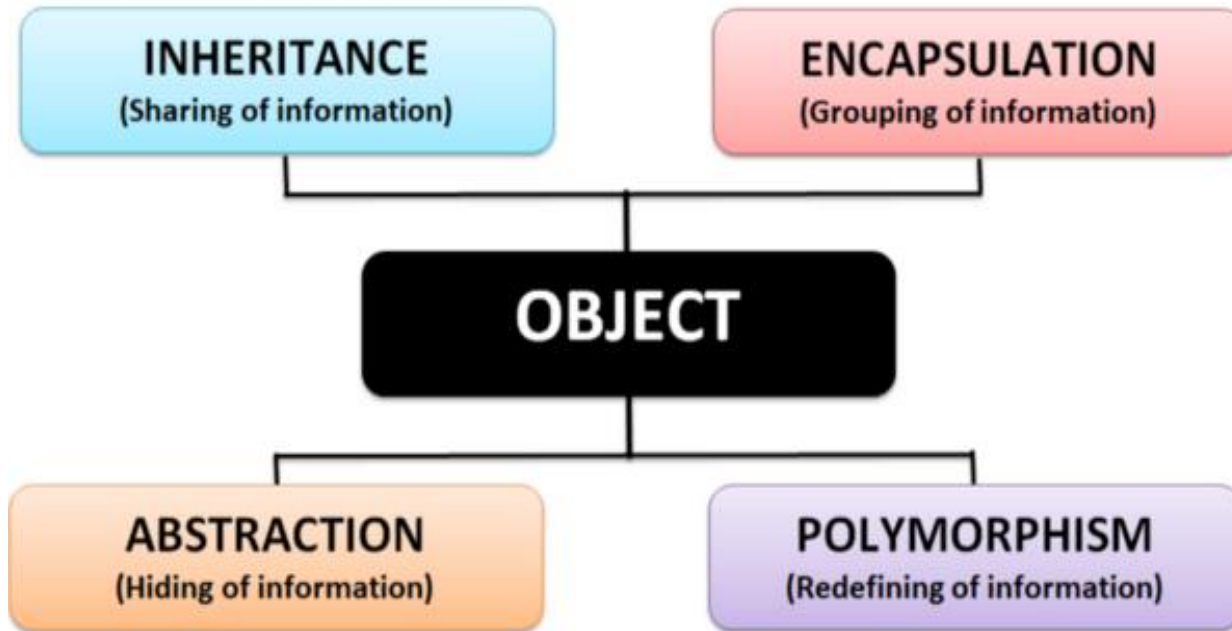
Hierarchical
Inheritance



The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub. The text is centered in the middle of the slide.

1 – What is Polymorphism?

Understanding the Concept of OOP



- ✓ Object Oriented Programming (OOP) atau pemrograman berorientasi objek adalah salah satu pendekatan paling efektif dalam menulis kode suatu program/software.
- ✓ Pada OOP, kita menulis class yang merepresentasikan suatu situasi seperti pada keadaan sebenarnya dan kita dapat membuat suatu objek berdasarkan class tersebut.

The four pillars of object-oriented programming in Python are: Abstraction; Encapsulation; Inheritance; Polymorphism.

What is Polymorphism?

- ❖ **Polymorphism** adalah kemampuan untuk mengambil bentuk yang berbeda. Polymorphism dalam Python memungkinkan kita untuk mendefinisikan metode pada child class dengan menggunakan nama yang sama seperti pada parent class.
- ❖ Polymorphism defines the ability to take different forms. Polymorphism in Python allows us to define methods in the child class with the same name as defined in their parent class.
- ❖ Polymorphism is the principle that one kind of thing can take a variety of forms. In the context of programming, this means that a single entity in a programming language can behave in multiple ways depending on the context.
- ❖ Polymorphism refers to having multiple forms. Polymorphism is a programming term that refers to the use of the same function name, but with different signatures, for multiple types.

Polymorphism digabung dari dua kata:

Poly = banyak

Morphism = bentuk (*form*)

} Fungsi yang sama dapat digunakan pada *tipe* yang berbeda.

Dapatkah kalian
memberikan contoh satu
jenis object/benda/orang
yang dapat mengambil
berbagai bentuk yang
berbeda?



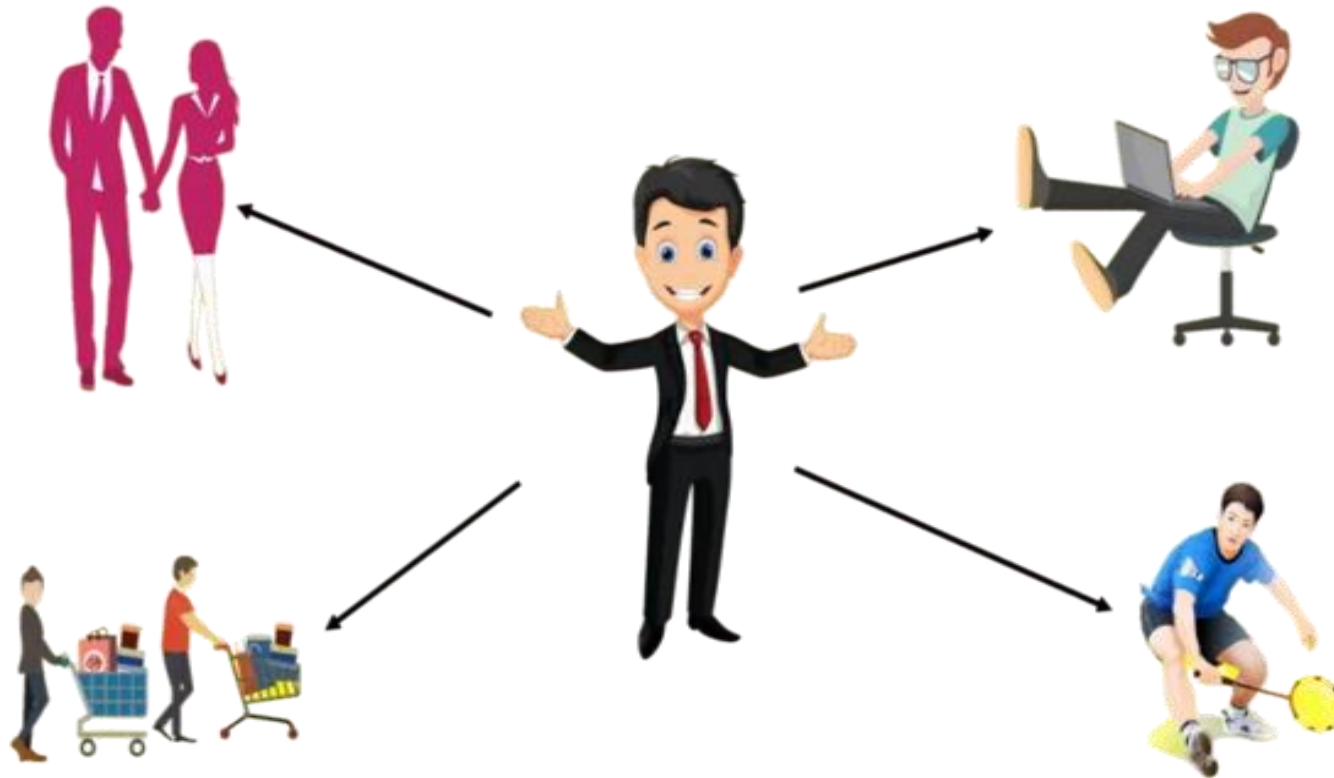
OOP Ilustrasi Polymorphism

Polymorphism adalah kemampuan untuk mengambil bentuk yang berbeda.



OOP Ilustrasi Polymorphism

Polymorphism adalah kemampuan untuk mengambil bentuk yang berbeda.



You can take different forms as per the situation.

OOP Ilustrasi Polymorphism

Polymorphism adalah kemampuan untuk mengambil bentuk yang berbeda.



Employee Role



Wife Role



Tennis Player Role

You can take different forms as per the situation.

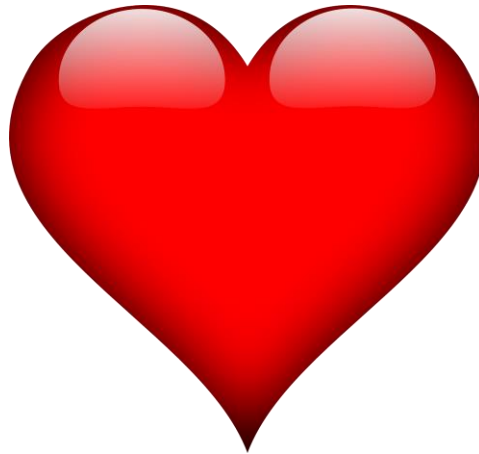
OOP Ilustrasi Polymorphism

Polymorphism adalah kemampuan untuk mengambil bentuk yang berbeda.



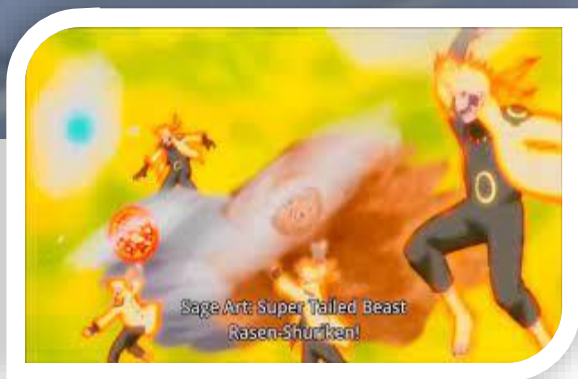
OOP Ilustrasi Polymorphism

Polymorphism adalah bukan sebuah kemampuan untuk mempunyai banyak pasangan.



Naruto kage bunshin no jutsu





Sage Art: Super Tailed Beast
Rasen-Shuriken!





Naruto
Version #1

Naruto
Version #2

Naruto
Version #3

Naruto
Version #4

What is Polymorphism?

Polymorphism can be defined as a **condition that occurs in many different forms**. It is a concept in Python programming wherein **an object defined in Python can be used in different ways**. It allows the programmer to define multiple methods in a derived class, and it has the same name as present in the parent class. Such scenarios support **method overloading** in Python.



What is Polymorphism?

Polymorphism adalah sebuah cara atau *ability* untuk mengambil bentuk yang berbeda. Dalam context ini, *polymorphism* memungkinkan kita untuk mendefinisikan *method* pada **child class** dengan menggunakan nama yang sama seperti pada **parent class**.

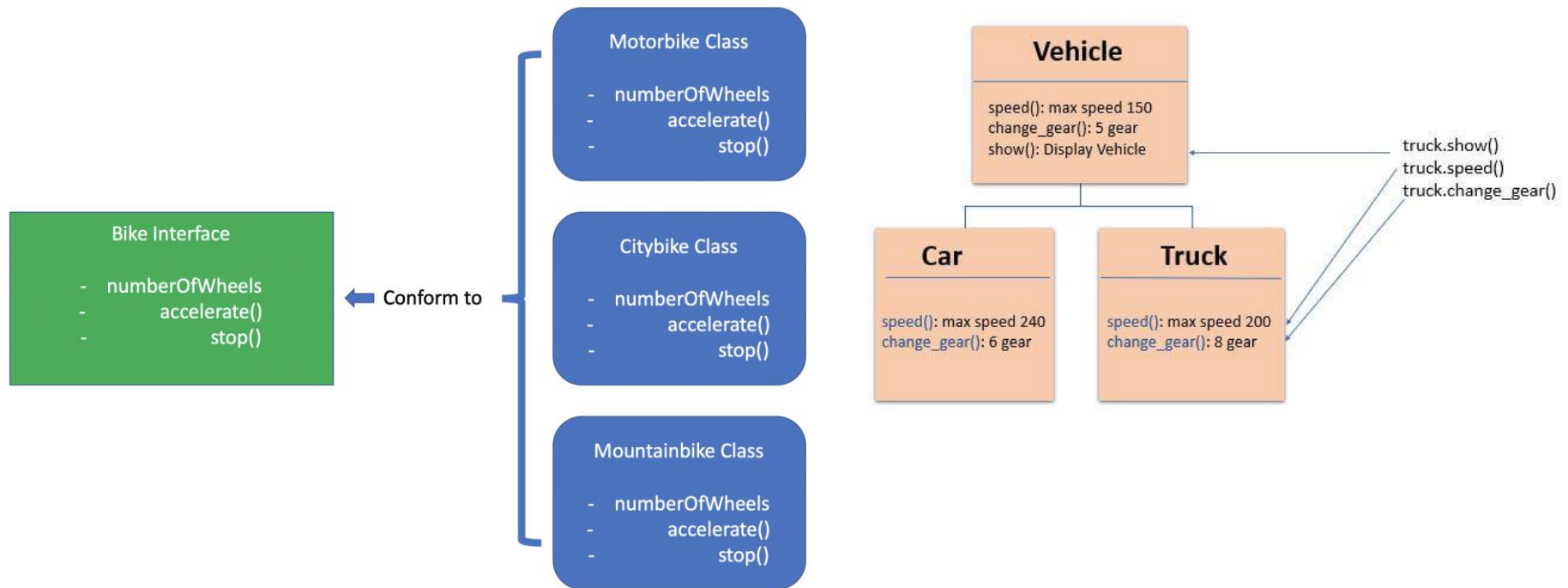


What is the relation between parent and child class?

A parent class contains the child class. Where as a derived class inherits from a base class. They are similar because the child (*or derived*) can access the parents (*or base*) properties and methods (*where allowed*). They are different because we can refer to a property of the child class in the form of Parent.

Polymorphism in Python

Polymorphism dalam Python memungkinkan kita untuk mendefinisikan **metode** pada *child class* dengan menggunakan nama yang sama seperti pada *parent class*.



Class child mewarisi seluruh method dari *parent class*. Tapi, ada beberapa kasus di mana **metode tersebut tidak cocok** dengan *child class*. Oleh karena itu, kita harus mengimplementasikan (*re-used*) kembali metode yang pada *child class* yang dinamakan **Method Overriding**.

[1] Polymorphism *with* Class

```
# create class 1
class Cat:
    # constructor (initialize instance variable)
    def __init__(self, nama, umur):
        self.nama = nama
        self.umur = umur

    def speak(self):
        print("Cat speaks Meow Meow Meow.")

# create class 2
class Dog:
    # constructor (initialize instance variable)
    def __init__(self, nama, umur):
        self.nama = nama
        self.umur = umur

    def speak(self):
        print("Dog speaks Gukk Gukk Gukk.")

# object of Cat created
kucing1 = Cat("Catty", 5)

# object of Dog created
anjing1 = Dog("Doggy", 4)

# memanggil metode tanpa mengabaikan objek
kucing1.speak()
anjing1.speak()

# memanggil metode dengan mengabaikan objek
for hewan in (kucing1, anjing1):
    hewan.speak()
```

- ❖ We can use the concept of polymorphism when creating methods for a class. Python allows different classes to use methods with the same name. We can then call the method by ignoring the object we are using.
- ❖ Kita dapat menggunakan konsep polymorphism ketika membuat metode sebuah Class.
- ❖ Bahasa pemrograman Python memungkinkan sebuah class yang berbeda untuk menggunakan metode dengan nama yang sama.
- ❖ Kemudian kita dapat memanggil metode tersebut dengan mengabaikan objek yang sedang kita gunakan.

[2] Polymorphism *with* Inheritance

```
# create parent class
class Bird:
    def intro(self):
        print("There are several different types of bird species.")

    def fly(self):
        print("Almost all birds can fly, but there are some that cannot.")

# create child class 1
class Eagle(Bird):
    def fly(self):
        print("Yes, an eagle can fly.")

# create child class 2
class Weka(Bird):
    def fly(self):
        print("No, an Weka can't fly.")

# # object of parent class & child class created
obj_burung = Bird()
obj_elang = Eagle()
obj_weka = Weka()

# user-defined function is called from obj_burung (parent class)
obj_burung.intro()
obj_burung.fly()

# user-defined function is called from obj_elang (child class)
obj_elang.intro()
obj_elang.fly()

# user-defined function is called from obj_weka (child class)
obj_weka.intro()
obj_weka.fly()
```



The diagram illustrates the inheritance hierarchy using red curly braces and numbers. A large brace labeled '1' groups the `Bird` class and its methods (`intro` and `fly`). A second brace labeled '2' groups the `Eagle` class and its `fly` method. A third brace labeled '3' groups the `Weka` class and its `fly` method. This indicates that `Eagle` inherits from `Bird`, and `Weka` inherits from `Bird`, with both child classes overriding the `fly` method.

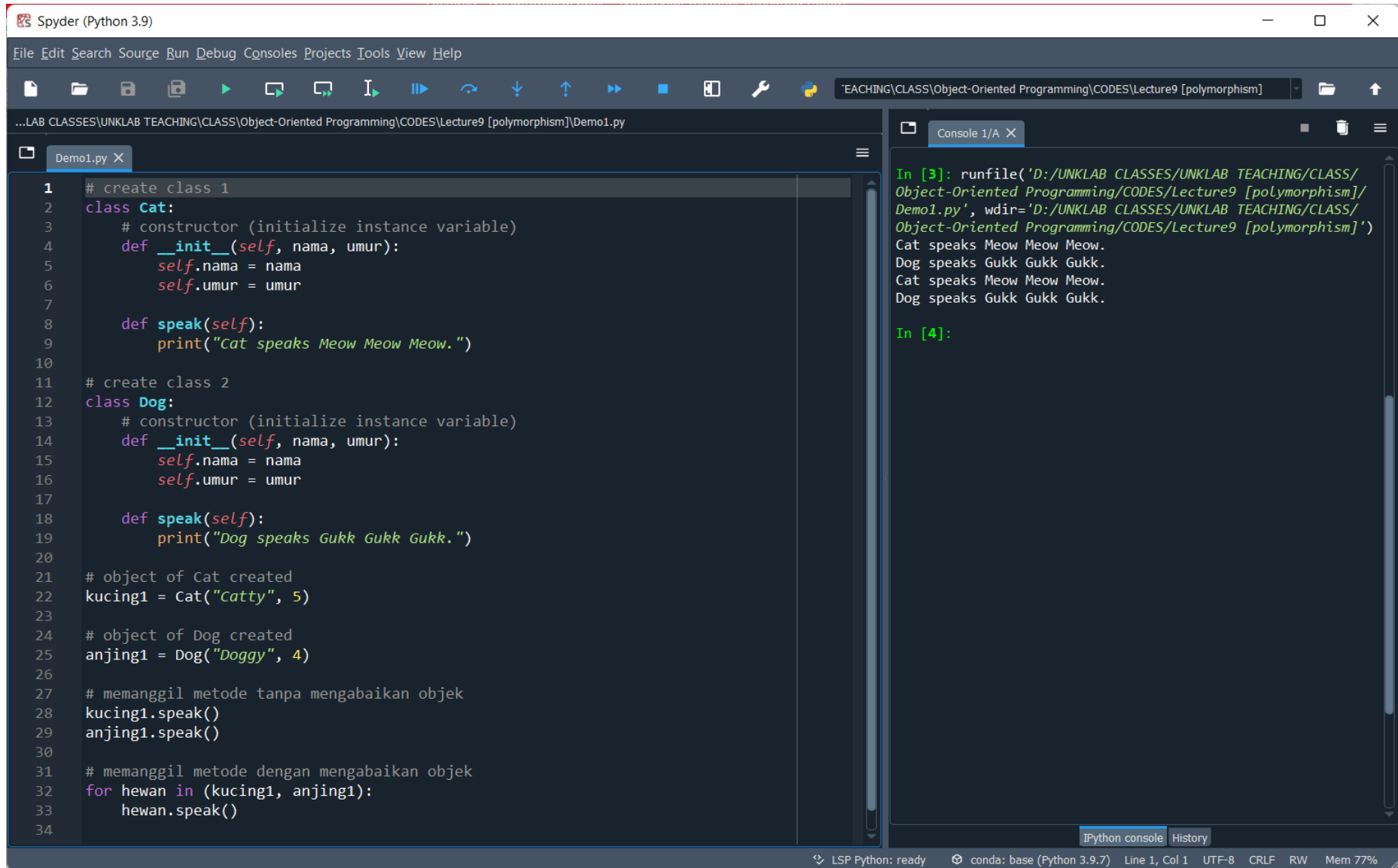
- ❖ Polymorphism in object-oriented programming with Python defines a *child class* that has the same method names as the *parent class*. In inheritance, the **child class** inherits methods from the **parent class**. In addition, **it is possible to modify methods** in the child class that have been inherited from the parent class.
- ❖ Polymorphism pada pemrograman berorientasi objek dengan Python mendefinisikan *child class* yang memiliki kesamaan nama metode pada *parent class*. Pada inheritance, *child class* mewarisi metode dari *parent class*.
- ❖ Selain itu, **memungkinkan untuk memodifikasi metode** di dalam child class yang telah diwarisi dari parent class.

Polymorphism is a concept where objects of different classes can have different behaviors when calling the same method.



Exercise *for* Students

Lab Exercise #1



The screenshot displays the Spyder Python IDE interface. The main editor window on the left contains a Python script named `Demo1.py`. The script defines two classes, `Cat` and `Dog`, each with an `__init__` constructor and a `speak` method. It then creates instances of these classes, `kucing1` and `anjing1`, and demonstrates calling the `speak` method both directly on the objects and using a `for` loop.

```
1 # create class 1
2 class Cat:
3     # constructor (initialize instance variable)
4     def __init__(self, nama, umur):
5         self.nama = nama
6         self.umur = umur
7
8     def speak(self):
9         print("Cat speaks Meow Meow Meow.")
10
11 # create class 2
12 class Dog:
13     # constructor (initialize instance variable)
14     def __init__(self, nama, umur):
15         self.nama = nama
16         self.umur = umur
17
18     def speak(self):
19         print("Dog speaks Gukk Gukk Gukk.")
20
21 # object of Cat created
22 kucing1 = Cat("Catty", 5)
23
24 # object of Dog created
25 anjing1 = Dog("Doggy", 4)
26
27 # memanggil metode tanpa mengabaikan objek
28 kucing1.speak()
29 anjing1.speak()
30
31 # memanggil metode dengan mengabaikan objek
32 for hewan in (kucing1, anjing1):
33     hewan.speak()
34
```

The right-hand pane shows the IPython console output. It displays the results of running the script, showing the output of the `speak` method for both the `Cat` and `Dog` objects. The console also shows the execution of the `runfile` command.

```
In [3]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture9 [polymorphism]/
Demo1.py', wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture9 [polymorphism]')
Cat speaks Meow Meow Meow.
Dog speaks Gukk Gukk Gukk.
Cat speaks Meow Meow Meow.
Dog speaks Gukk Gukk Gukk.

In [4]:
```

The bottom status bar indicates the current state of the IDE: "LSP Python: ready", "conda: base (Python 3.9.7)", "Line 1, Col 1", "UTF-8", "CRLF", "RW", and "Mem 77%".

Lab Exercise #2

The screenshot displays the Spyder Python IDE interface. The main editor window shows a Python script named `Demo2.py` with the following code:

```
1 # create parent class
2 class Bird:
3     def intro(self):
4         print("There are several different types of bird species.")
5
6     def fly(self):
7         print("Almost all birds can fly, but there are some that cannot.")
8
9 # create child class 1
10 class Eagle(Bird):
11
12     def fly(self):
13         print("Yes, an eagle can fly.")
14
15 # create child class 2
16 class Weka(Bird):
17
18     def fly(self):
19         print("No, an Weka can't fly.")
20
21 # # object of parent class & child class created
22 obj_burung = Bird()
23 obj_elang = Eagle()
24 obj_weka = Weka()
25
26 # user-defined function is called from obj_burung (parent class)
27 obj_burung.intro()
28 obj_burung.fly()
29
30 # user-defined function is called from obj_elang (child class)
31 obj_elang.intro()
32 obj_elang.fly()
33
34 # user-defined function is called from obj_weka (child class)
35 obj_weka.intro()
36 obj_weka.fly()
```

The IPython console on the right shows the execution output:

```
In [4]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture9 [polymorphism]/
Demo2.py', wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture9 [polymorphism]')
There are several different types of bird species.
Almost all birds can fly, but there are some that cannot.
There are several different types of bird species.
Yes, an eagle can fly.
There are several different types of bird species.
No, an Weka can't fly.

In [5]:
```

The status bar at the bottom indicates the environment is LSP Python: ready, conda: base (Python 3.9.7), and the current position is Line 1, Col 1. The memory usage is 81%.

Discuss, answer questions and 6 students will be asked to participate in the class.

- 1) How is the concept of polymorphism applied in Demo2.py code? (Bagaimana konsep polymorphism diterapkan dalam kode Demo2.py di atas?)
- 2) From three classes in Demo2.py file, choose which one is the *parent class* and which one is the *child class*? Why? (Dari tiga class yang ada pada file Demo2.py, pilihlah kelas mana yang merupakan kelas induk dan kelas mana yang merupakan kelas turunan? Mengapa?)
- 3) What is the main difference between the **fly() method** in the parent class Bird and the child class Eagle? (Apa perbedaan utama antara *fly() method* dalam parent class Bird dan child class Eagle?)
- 4) Why does the **fly() method** in the child class Weka have different behavior from the **fly() method** in the parent class Bird? (Mengapa *fly() method* dalam *child class Weka* memiliki perilaku yang berbeda dari *fly() method* dalam *parent class Bird*?)
- 5) Why does the child class Eagle override the *fly() method* of the parent class Bird? What are its benefits in the context of polymorphism? (Mengapa child class Eagle menempa metode fly dari parent class Bird? Apa manfaatnya dalam konteks polymorphism?)
- 6) What happens when *intro() method* is called from objects of different classes, such as *obj_bird*, *obj_eagle*, and *obj_weka*? (Apa yang terjadi ketika *intro() method* dipanggil dari objek-objek kelas yang berbeda, seperti *obj_burung*, *obj_elang*, dan *obj_weka*?)

URL:

<https://wheelofnames.com/>

The screenshot displays the 'Wheel of Names' website. The browser's address bar shows the URL 'https://wheelofnames.com/'. The website's header includes a navigation bar with options: New, Open, Save, Share, Gallery, Customize, More, and English. The main content area features a large, colorful spinning wheel with 29 segments, each containing a name. Overlaid on the wheel is the text 'Click to spin' and 'or press ctrl+enter'. To the right of the wheel is a panel titled 'Entries 29' and 'Results 0'. This panel includes controls for 'Shuffle' and 'Sort', an 'Add image' button, and an 'Advanced' checkbox. Below these controls is a scrollable list of 29 names. At the bottom of the panel, it shows 'Version 284' and a link to the 'Changelog'.

Entries 29 Results 0

Shuffle Sort

Add image Advanced

, Jeremy Timothy
-, Immanuel Gloria Griffin
Andolo, Cathleen Yulina
Andolo, Clara Patricia
Hong Joyo, Zack Lee
Johanes Fijey
Langi, Carolina Pears Pamela
Lolong, Deeva Civia Aulia
Londah, Lana Lois Lane
Mait, Swarsh Timoti Rezki
Mulait, Cristian Skrivent
Oroh, Injilio
Parangka, Cavin Adam
Patras, Samuel Christian
Paulus, Deaven Antonio

Version 284 [Changelog](#)

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

